Sohaa I Jamadar | 1252090024 | SEDA(AI) | 24

```
import pandas as pd;
df=pd.read_csv("uber.csv")
df
```

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	drop
0	24238194	2015-05-07 19:52:06.0000003	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	-73.999512	
1	27835199	2009-07-17 20:04:56.0000002	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	-73.994710	
2	44984355	2009-08-24 21:45:00.00000061	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	-73.962565	
3	25894730	2009-06-26 08:22:21.0000001	5.3	2009-06-26 08:22:21 UTC	-73.976124	40.790844	-73.965316	
4	17610152	2014-08-28 17:47:00.000000188	16.0	2014-08-28 17:47:00 UTC	-73.925023	40.744085	-73.973082	
...	...	...	...	...	...	...	...	
17843	52767148	2012-04-14 02:47:00.000000111	18.1	2012-04-14 02:47:00 UTC	-73.949998	40.780385	-74.005577	
17844	32667836	2009-02-19 07:25:00.000000153	26.5	2009-02-19 07:25:00 UTC	-73.982980	40.762627	-73.872748	
17845	37364579	2012-05-04 03:54:12.0000001	4.9	2012-05-04 03:54:12 UTC	-73.962850	40.804438	-73.952757	
17846	16137370	2014-05-14 16:35:21.0000003	11.0	2014-05-14 16:35:21 UTC	-73.987277	40.738977	-73.976124	
17847	42510306	2009-05-06 08:31:56.0000	NaN	NaN	NaN	NaN	NaN	

17848 rows × 9 columns

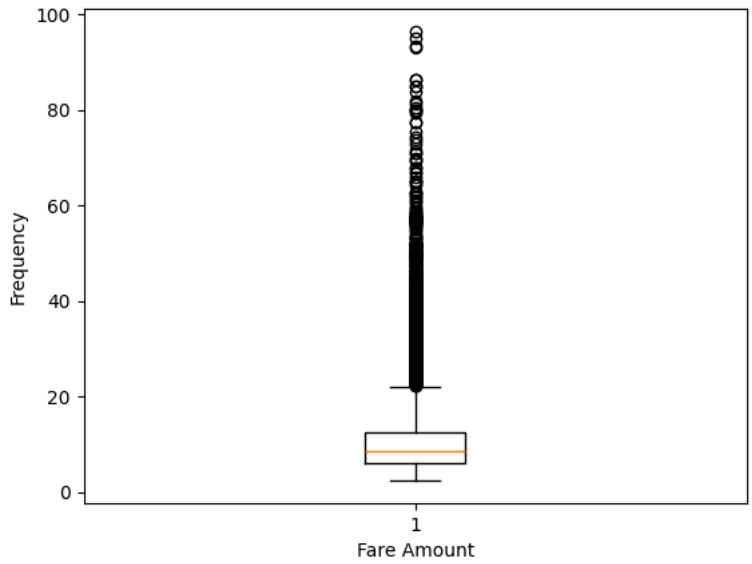
Next steps: Generate code with df New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17848 entries, 0 to 17847
Data columns (total 9 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Unnamed: 0         17848 non-null  int64
 1   key                17848 non-null  object
 2   fare_amount        17847 non-null  float64
 3   pickup_datetime    17847 non-null  object
 4   pickup_longitude   17847 non-null  float64
 5   pickup_latitude    17847 non-null  float64
 6   dropoff_longitude  17847 non-null  float64
 7   dropoff_latitude   17847 non-null  float64
 8   passenger_count    17847 non-null  float64
dtypes: float64(6), int64(1), object(2)
memory usage: 1.2+ MB
```

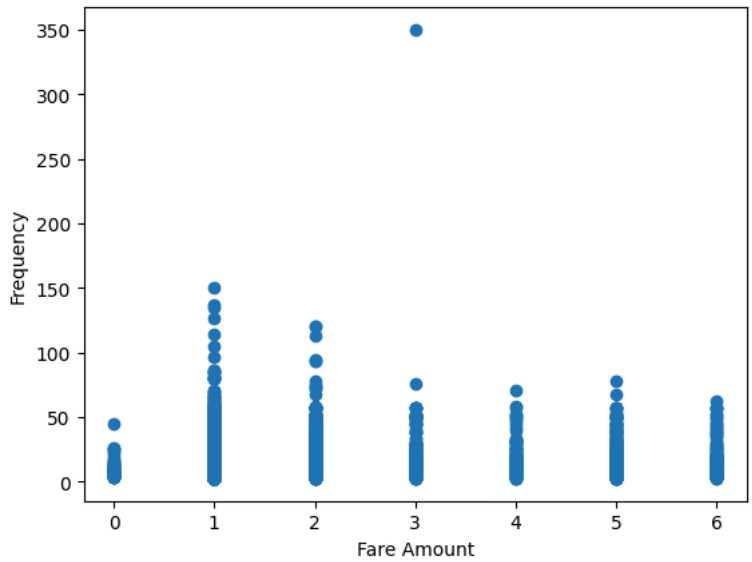
```
import matplotlib.pyplot as plt

fare_amountFilter=df[(df['fare_amount']>0)&(df['fare_amount']<100)]
plt.boxplot(fare_amountFilter['fare_amount'])
plt.xlabel('Fare Amount')
plt.ylabel('Frequency')
plt.show()
```




```
import matplotlib.pyplot as plt

plt.scatter(df['passenger_count'],df['fare_amount'])
plt.xlabel('Fare Amount')
plt.ylabel('Frequency')
plt.show()
```




```
import numpy as np

q1=np.percentile(df['fare_amount'],25)
q2=np.percentile(df['fare_amount'],50)
q3=np.percentile(df['fare_amount'],75)

IQR=q3-q1
lower_bound=q1-(1.5*IQR)
upper_bound=q3+(1.5*IQR)

print("Q1: ", q1)
print("Q2: ",q2)
print("Q3: ",q3)


print("IQR: ",IQR)
print("Lower bound: ",lower_bound)
print("Upper bound: ",upper_bound)
```

```
Q1:  nan
Q2:  nan
Q3:  nan
IQR:  nan
Lower bound:  nan
Upper bound:  nan
```

```
corr=df['fare_amount'].corr(df['passenger_count'])
print("correlation: ",corr)
corr
```

```
correlation:  0.008790635922843721
np.float64(0.008790635922843721)
```

Linear Regression

```
import pandas as pd
sample={
    "x":[2,4,6],
    "y":[50,55,65]
})

dff=pd.DataFrame(sample)
dff
```

	x	y
0	2	50
1	4	55
2	6	65

Next steps: Generate code with dff New interactive sheet

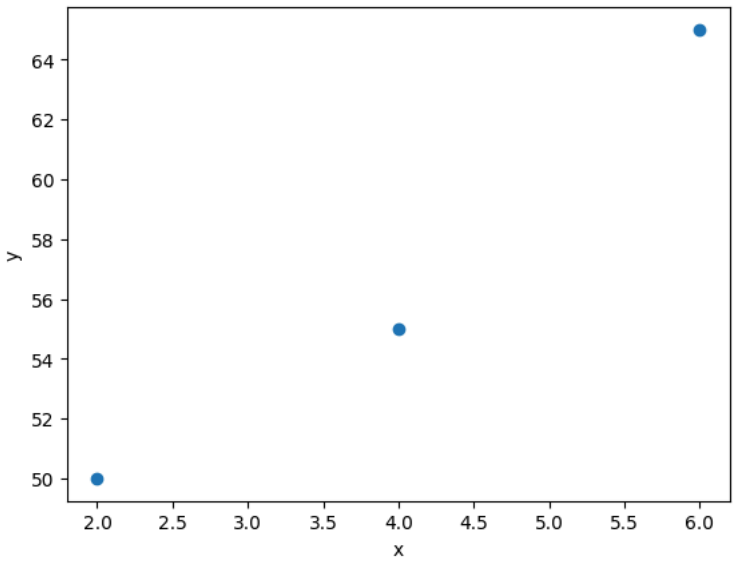
Linear Regression Model

```
from sklearn.linear_model import LinearRegression
lr=LinearRegression()
lr.fit(dff[['x']],dff['y'])
y_pred=lr.predict(dff[['x']])
print(y_pred)
```

```
[49.16666667 56.66666667 64.16666667]
```

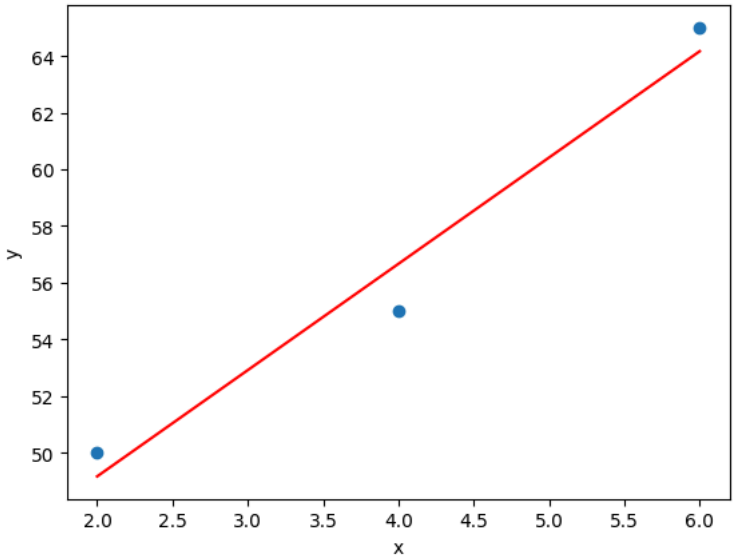
Scatter Plot

```
import matplotlib.pyplot as plt
plt.scatter(dff['x'],dff['y'])
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```



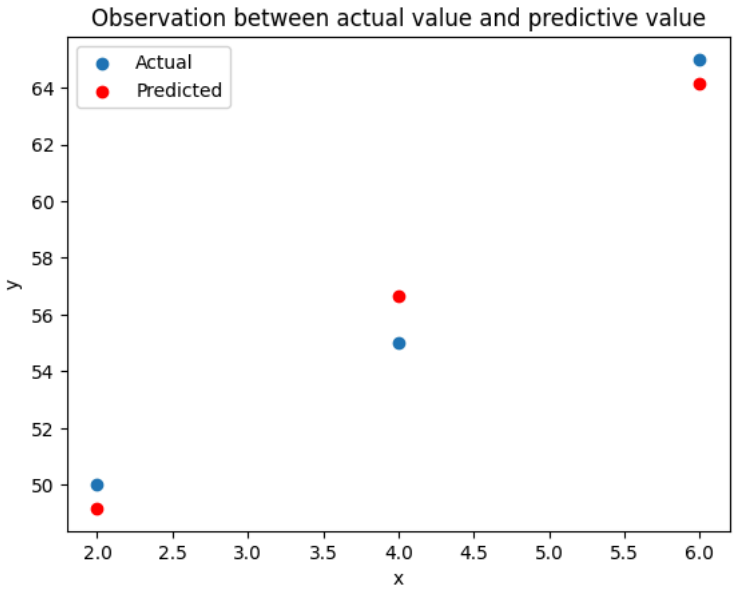

```
import matplotlib.pyplot as plt
plt.scatter(dff['x'],dff['y'])
plt.plot(dff['x'],y_pred,color="red")
plt.xlabel('x')
plt.ylabel('y')
plt
```

```
<module 'matplotlib.pyplot' from '/usr/local/lib/python3.12/dist-packages/matplotlib/pyplot.py'>
```




```
import matplotlib.pyplot as plt
plt.scatter(dff['x'],dff['y'])
plt.scatter(dff['x'],y_pred,color="red")
plt.xlabel('x')
plt.ylabel('y')
plt.title('Observation between actual value and predictive value')
plt.legend(['Actual','Predicted'])
plt
```

```
<module 'matplotlib.pyplot' from '/usr/local/lib/python3.12/dist-packages/matplotlib/pyplot.py'>
```




```
sample2=({
    "rainfall":[12.3,13.7,14.5,11.2,13.2,14.1,12.0],
    "yield":[6.25,8.92,8.24,5.27,7.21,8.71,5.68]
})

df2=pd.DataFrame(sample2)
df2
```

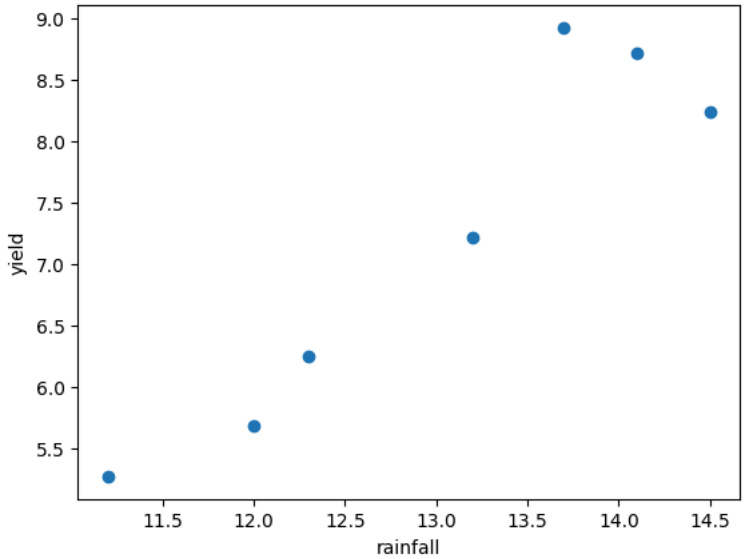
	rainfall	yield
0	12.3	6.25
1	13.7	8.92
2	14.5	8.24
3	11.2	5.27
4	13.2	7.21
5	14.1	8.71
6	12.0	5.68

Next steps: Generate code with df2 New interactive sheet

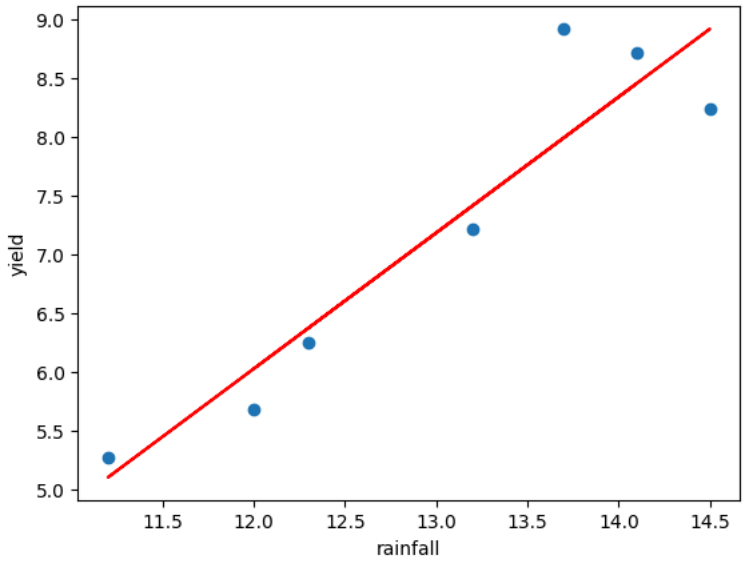
```
from sklearn.linear_model import LinearRegression
lr=LinearRegression()
lr.fit(df2[['rainfall']],df2['yield'])
predicted_yield=lr.predict(df2[['rainfall']])
print(predicted_yield)
```

```
[6.37320118 7.99251311 8.91783421 5.10088467 7.41418742 8.45517366
 6.02620577]
```

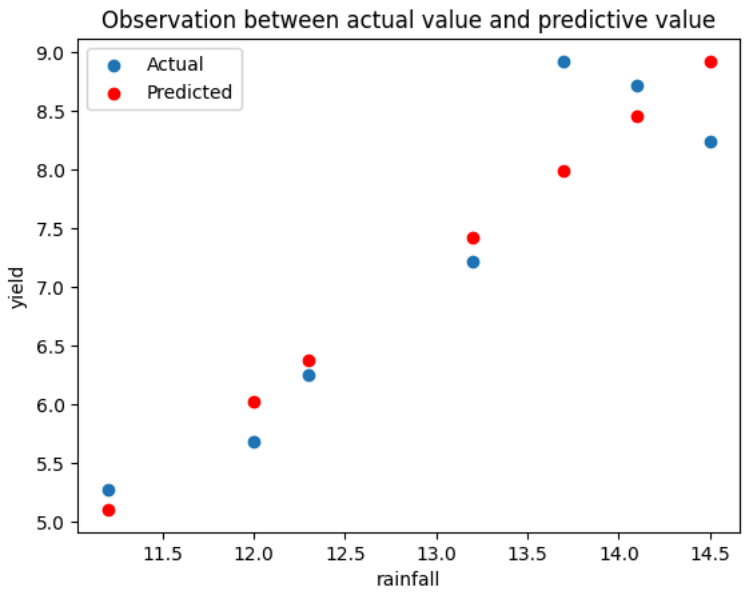
```
plt.scatter(df2['rainfall'],df2['yield'])
plt.xlabel('rainfall')
plt.ylabel('yield')
plt.show()
```




```
plt.scatter(df2['rainfall'],df2['yield'])
plt.plot(df2['rainfall'],predicted_yield,color="red")
plt.xlabel('rainfall')
plt.ylabel('yield')
plt.show()
```




```
plt.scatter(df2['rainfall'],df2['yield'])
plt.scatter(df2['rainfall'],predicted_yield,color="red")
plt.xlabel('rainfall')
plt.ylabel('yield')
plt.legend(['Actual','Predicted'])
plt.title('Observation between actual value and predictive value')
plt.show()
```




```
slop=lr.coef_[0]
intercept=lr.intercept_
print("slop: ",slop)
print("intercept: ",intercept)
```

```
slop:  1.1566513761467894
intercept:  -7.853610747051119
```

Confusion Matrix and Classification Report

```
from sklearn.metrics import confusion_matrix,classification_report
actual=[1,0,0,1,0,0,1,0,0,1]
predictive=[1,0,0,1,0,0,0,1,0,0]
matrix=confusion_matrix(actual,predictive,labels=[1,0])
print("Confusion matrix: \n",matrix)
```

```
Confusion matrix:
 [[2 2]
 [1 5]]
```

```
tp,fn,fp,tn=confusion_matrix(actual,predictive,labels=[1,0]).reshape(-1)
print('Outcome value:\n',tp,fn,fp,tn)
```

```
matrix=classification_report(actual,predictive,labels=[1,0])
print("Classification report: \n",matrix)
```

```
Outcome value:
 2 2 1 5
Classification report:
               precision    recall  f1-score   support

           1       0.67      0.50      0.57         4
           0       0.71      0.83      0.77         6

    accuracy                           0.70        10
   macro avg       0.69      0.67      0.67        10
weighted avg       0.70      0.70      0.69        10
```